



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机组成原理实验报告

智能矩阵乘法优化挑战

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：王刚

2025 年 5 月 25 日

摘要

本文针对大规模矩阵乘法计算的性能瓶颈，系统性地比较和实现了包括 OpenMP、Block Parallel、SIMD、MPI 和基于曙光 DCU 加速器的 HIP 等多种优化方案。通过理论分析与实验评测，全面考察了各方案在运行时间、加速比、资源占用和 GFLOPS 等方面的表现。结果显示，DCU-HIP 方案在保证数值正确性的前提下，能够极大提升计算效率，实现超 5000 倍的加速比和远超 CPU 端的浮点运算性能。本文还借鉴了 PyTorch 等主流深度学习框架的高效 GEMM 优化思想，集成了分块、共享内存、访存模式等多项高性能计算技术，提出了适用于国产加速器的矩阵乘法最终优化方案。最后，分析了各方法的适用场景及存在的问题，并展望了未来的优化方向与工程应用前景。

关键字：大规模矩阵乘法；高性能计算；并行优化；HIP；DCU 加速器；PyTorch；共享内存；分块算法

目录

一、 引言	1
二、 理论基础	1
(一) 矩阵乘法基本原理	1
(二) 性能瓶颈分析	1
(三) 常见优化方法简介	1
三、 实验环境与工具	2
四、 算法实现	2
(一) 标准矩阵乘法	2
(二) 优化算法设计与实现	3
1. OpenMP 多线程并行优化	3
2. 子块并行优化	3
3. MPI 多进程并行优化	4
4. SIMD 加速优化	4
5. DCU/HIP 加速优化	5
五、 实验结果与性能分析	6
(一) 运行时间对比	6
(二) 资源占用分析	6
(三) GFLOPS 分析	7
六、 问题讨论	8
(一) 各优化方法优缺点分析	8
(二) 适用场景讨论	9
(三) 影响性能的关键因素	9
(四) 实验中遇到的问题及解决办法	9

七、 结合多种优化的最终方案	9
(一) 设计与实现要点	9
(二) 算法实现	10
(三) 运行结果分析	10
(四) 进一步优化方向	11
八、 结论与展望	11
(一) 优化效果总结	11
(二) 后续改进方向与应用前景	12

一、引言

矩阵乘法是科学计算、工程模拟、人工智能等众多领域中的基础运算，其高效实现直接影响到相关应用的整体性能。随着大数据和人工智能的快速发展，涉及大规模矩阵运算的场景日益增多，如何高效地完成大规模矩阵乘法成为当前计算领域的重要研究课题。本实验旨在实现标准的矩阵乘法算法，并探索多种优化技术以提升其计算效率。通过理论分析和实验对比，系统评估不同优化方法的性能表现，为大规模矩阵计算的高效实现提供参考。

二、理论基础

(一) 矩阵乘法基本原理

设有矩阵 A ($N \times M$) 和矩阵 B ($M \times P$)，其乘积矩阵 C 为 $N \times P$ 矩阵。 C 的每个元素 c_{ij} 计算公式为：

$$c_{ij} = \sum_{k=1}^M a_{ik} \cdot b_{kj}, \quad 1 \leq i \leq N, 1 \leq j \leq P \quad (1)$$

标准的矩阵乘法算法的时间复杂度为 $O(NMP)$ 。对于大规模矩阵，这一运算复杂度将带来巨大的计算和存储压力。

(二) 性能瓶颈分析

传统的三重循环实现方式在计算大规模矩阵时主要面临以下性能瓶颈：

- **计算量大**：元素级别的逐一计算，导致运算次数随矩阵规模迅速增长。
- **内存访问局部性差，缓存未命中率高**：标准三重循环中，若访问矩阵 B 的列，可能导致多次远距离跳转，内存空间局部性差，缓存利用率低，频繁访问主存，极大拖慢整体速度。
- **缺乏并行性**：标准实现未充分利用多核、多线程和加速器等现代计算资源。

(三) 常见优化方法简介

为提升矩阵乘法的性能，常用的优化方法包括但不限于：

- **多线程并行化 (OpenMP)**：利用多核 CPU 资源，将矩阵计算任务划分给不同线程并行执行。
- **子块并行 (Block-wise)**：将矩阵划分为子块，减少缓存未命中，并结合并行机制提升效率。
- **多进程并行 (MPI)**：适用于分布式或多节点系统，通过进程间协作完成大规模矩阵乘法。
- **SIMD 向量化 (SIMD)**：利用单指令多数据 (SIMD) 指令集，在单核内实现数据级并行，加速矩阵元素的批量乘加运算。
- **异构加速 (DCU/HIP)**：利用国产高性能加速器（如 DCU/GPU）与 HIP 编程模型实现大规模并行计算。

三、 实验环境与工具

本实验在曙光 DCU 实训平台上完成，具体配置如下：

- **编程语言：** C++
- **并行/加速开发框架：** 曙光 DCU ToolKit (DTK)
- **硬件环境：**
 - 8 核心 CPU
 - 1 张 DCU 加速卡
 - 16 GB 内存
- **运行环境：** 曙光 DCU 实训平台

所有性能测试与资源占用监控均在上述环境下完成，保证了各实现方案的可比性和实验结果的公平性。

四、 算法实现

(一) 标准矩阵乘法

标准矩阵乘法采用三重循环实现。对于矩阵 A (N×M) 和 B (M×P)，输出矩阵 C (N×P)，每个元素 c_{ij} 按如下公式计算：

$$c_{ij} = \sum_{k=1}^M a_{ik} \cdot b_{kj}$$

整体流程为：外层循环遍历 A 的每一行，中间循环遍历 B 的每一列，内层循环遍历对应的 k，并累加乘积。

标准矩阵乘法

```
1 void matmul_baseline(const std::vector<double> &A,  
2                      const std::vector<double> &B,  
3                      std::vector<double> &C, int N, int M, int P)  
4 {  
5     for (int i = 0; i < N; ++i)  
6         for (int j = 0; j < P; ++j)  
7             {  
8                 C[i * P + j] = 0;  
9                 for (int k = 0; k < M; ++k)  
10                    C[i * P + j] += A[i * M + k] * B[k * P + j];  
11             }  
12 }
```

(二) 优化算法设计与实现

1. OpenMP 多线程并行优化

标准三重循环实现未利用多核 CPU 资源。OpenMP [1] 多线程优化通过将最外层循环并行化，使多个线程可同时处理不同的矩阵行，大幅提升计算效率。

只需在外层循环加 ‘#pragma omp parallel for’，即可自动利用多线程并行。

OpenMP 多线程并行优化算法

```
1 void matmul_omp(const std::vector<double> &A,
2               const std::vector<double> &B,
3               std::vector<double> &C, int N, int M, int P)
4 {
5     #pragma omp parallel for
6     for (int i = 0; i < N; ++i)
7         for (int j = 0; j < P; ++j)
8             {
9                 double sum = 0.0;
10                for (int k = 0; k < M; ++k)
11                    sum += A[i * M + k] * B[k * P + j];
12                C[i * P + j] = sum;
13            }
14 }
```

2. 子块并行优化

标准矩阵乘法访问模式导致缓存局部性差，频繁跨行/跨列跳转，造成 cache 未命中。子块并行 (Block Tiling) 将矩阵划分为小块，每次只处理部分数据，提高缓存命中率，减少内存带宽压力。合理选择 block_size (如 32、64、128) 可兼容不同 CPU 缓存结构，达到最佳性能。

子块并行优化算法

```
1 void matmul_block_tiling(const std::vector<double> &A,
2                          const std::vector<double> &B,
3                          std::vector<double> &C, int N, int M, int P, int
4                          block_size = 64)
5 {
6     for (int ii = 0; ii < N; ii += block_size)
7         for (int jj = 0; jj < P; jj += block_size)
8             for (int kk = 0; kk < M; kk += block_size)
9                 for (int i = ii; i < std::min(ii + block_size, N); ++i)
10                    for (int j = jj; j < std::min(jj + block_size, P); ++j)
11                        {
12                            double sum = 0.0;
13                            for (int k = kk; k < std::min(kk + block_size, M); ++k)
14                                sum += A[i * M + k] * B[k * P + j];
15                            C[i * P + j] += sum;
16                        }
17 }
```

3. MPI 多进程并行优化

MPI 优化 [2] 将大矩阵按行分块分配至多台节点或多进程，打破单机内存和计算瓶颈。每个进程独立计算负责的部分，主进程收集各个结果，适合分布式和大规模并行场景。

MPI 多进程并行优化算法核心代码

```

1 void matmul_mpi(int N, int M, int P)
2 {
3     int rank, size;
4     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5     MPI_Comm_size(MPI_COMM_WORLD, &size);
6
7     int rows_per_proc = N / size;
8     int remainder = N % size;
9     int local_rows = rows_per_proc + (rank < remainder ? 1 : 0);
10
11     std::vector<double> local_A(local_rows * M);
12     std::vector<double> B(M * P);
13     std::vector<double> local_C(local_rows * P, 0);
14
15     // A/B的生成与分发
16     // B广播
17     // ...
18
19     // 本地计算
20     for (int i = 0; i < local_rows; ++i)
21         for (int j = 0; j < P; ++j)
22             for (int k = 0; k < M; ++k)
23                 local_C[i * P + j] += local_A[i * M + k] * B[k * P + j];
24
25     // 主进程收集C子块和结果验证
26     // ...
27 }

```

4. SIMD 加速优化

传统的矩阵乘法为三重循环，CPU 一次只处理一对元素，效率较低。利用 SIMD 指令（如 AVX2、SVE 等），每条指令可并行处理多个数据元素（如 4 个 double 或 8 个 float），极大提升运算效率。

向量化的关键在于将数据按内存对齐批量加载，并用 SIMD 指令进行批量乘加，实现数据级并行。

MPI 多进程并行优化算法核心代码

```

1 void matmul_simd(const std::vector<double> &A,
2                 const std::vector<double> &B,
3                 std::vector<double> &C, int N, int M, int P)
4 {
5     // 将B转置，方便SIMD按行访问

```

```

6      std::vector<double> B_T(P * M);
7      for (int i = 0; i < M; ++i)
8          for (int j = 0; j < P; ++j)
9              B_T[j * M + i] = B[i * P + j];
10
11     for (int i = 0; i < N; ++i)
12     {
13         for (int j = 0; j < P; ++j)
14         {
15             __m256d sum_vec = _mm256_setzero_pd();
16             int k = 0;
17             // 每次处理4个double
18             for (; k + 4 <= M; k += 4)
19             {
20                 __m256d a_vec = _mm256_loadu_pd(&A[i * M + k]);
21                 __m256d b_vec = _mm256_loadu_pd(&B_T[j * M + k]);
22                 sum_vec = _mm256_fmadd_pd(a_vec, b_vec, sum_vec); // sum_vec
23                                     += a_vec * b_vec
24             }
25             // 水平加和sum_vec
26             double sum_arr[4];
27             _mm256_storeu_pd(sum_arr, sum_vec);
28             double sum = sum_arr[0] + sum_arr[1] + sum_arr[2] + sum_arr[3];
29             // 处理剩余的
30             for (; k < M; ++k)
31                 sum += A[i * M + k] * B_T[j * M + k];
32             C[i * P + j] = sum;
33         }
34     }

```

5. DCU/HIP 加速优化

本方法采用了国产高性能异构加速器——曙光 DCU 进行加速，针对 AI 训练、推理及高性能计算等应用场景，显著提升了计算效率。DCU 具备与 NVIDIA GPU 类似的大规模并行计算能力，能够高效处理矩阵乘法等密集型计算任务。

在技术实现上，本方法基于 HIP C++ 编程接口开发。HIP 由 AMD 提出，兼容 CUDA 编程模型，能够方便地将 CUDA 代码迁移至国产 DCU 平台，降低开发和适配成本，实现跨平台的高性能计算。

采用 DCU 加速的主要优势包括：

- 充分利用 DCU 的大规模并行处理能力，极大提升数据处理和训练速度；
- 利用高带宽片上存储和共享内存，有效优化数据访问效率，减少内存瓶颈；
- 通过 HIP 接口，便于现有 CUDA 项目的快速迁移和国产化适配；
- 支持与 CPU 端 OpenMP/MPI 等并行技术协同，实现异构多级并行优化。

DCU/HIP 加速优化算法


```

1  __global__ void matmul_kernel(const double *A, const double *B, double *C,
2      int n, int m, int p)
3  {
4      int row = blockIdx.y * blockDim.y + threadIdx.y;
5      int col = blockIdx.x * blockDim.x + threadIdx.x;
6      if (row < n && col < p)
7      {
8          double sum = 0.0;
9          for (int k = 0; k < m; ++k)
10             sum += A[row * m + k] * B[k * p + col];
11         C[row * p + col] = sum;
12     }
13 }

```

五、实验结果与性能分析

(一) 运行时间对比

这里我们在矩阵乘法计算规模为 $N = 1024$, $M = 2048$, $P = 512$ 的情况下进行性能测试。

可以看到, 不同的加速方法均显著提升了计算性能。其中, OpenMP 方案的加速比为 11.66 倍, Block Parallel 方案的加速比为 1.40 倍, SIMD 方案的加速比为 4.36 倍, MPI 方案的加速比为 5.97 倍。而 DCU-HIP 方案则表现出色, 运行时间仅为 0.0033 秒, 达到了 5555.43 倍的加速比。

算法方案	运行时间 (秒)	加速比
Baseline (朴素)	18.318	1.00
OpenMP	1.571	11.66
Block Parallel	13.077	1.40
SIMD	4.205	4.36
MPI	3.067	5.97
DCU-HIP	0.0033	5555.43

表 1: 不同矩阵乘法实现方案运行时间与加速比

(二) 资源占用分析

我们对比分析了 Baseline (朴素实现)、OpenMP、Block Parallel、SIMD、MPI 和 HIP/DCU (加速器) 等多种矩阵乘法实现方案的资源占用情况。主要关注 CPU 利用率、主机内存峰值以及加速器 (GPU/DCU) 显存与利用率等核心指标。各方案的详细资源占用数据见表2。

方案	CPU 利用率 (%)	最大内存占用 (MB)	加速器显存 (MB)
Baseline	99	34.5	—
OpenMP	152	34.6	—
Block Parallel	99	34.5	—
SIMD	99	42.5	—
MPI	383	45.1	—
HIP/DCU	99	169.2	10

表 2: 各方案资源占用对比

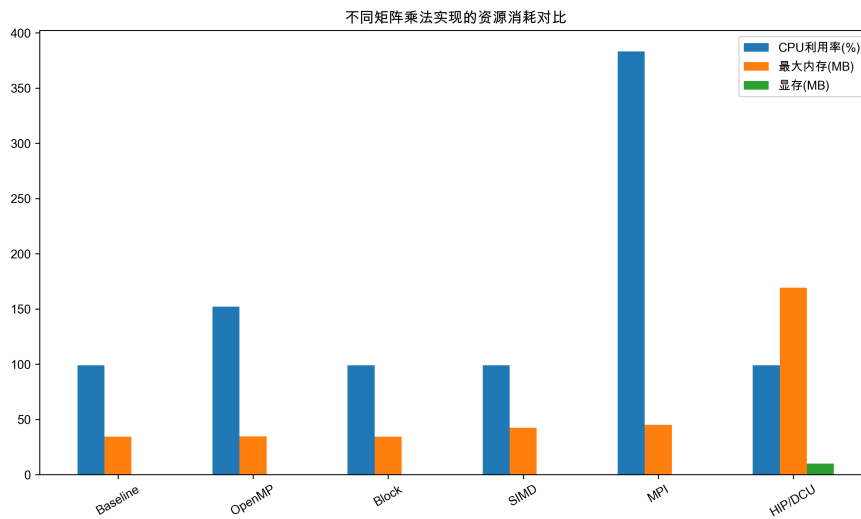


图 1: 资源占用对比

从 CPU 利用率上看, Baseline、Block Parallel 和 SIMD 方案均为 99%, 说明主要在单核或单线程下运行。OpenMP 和 MPI 方案则充分利用了多核并行能力, CPU 利用率分别达到 152% 和 383%, 显著提升了处理器资源的利用率, 体现了其并行加速的优势。

在主机内存占用方面, 除 SIMD 与 MPI 略高外, 其余 CPU 端方案最大内存占用均在 35MB 左右, 差异较小。值得注意的是, HIP/DCU 方案的主机内存占用显著提升, 达到约 169MB。这主要由于需要在主机和加速卡之间进行数据交互和缓存, 尤其是在大规模数据传输时会有额外开销。

综合来看, OpenMP 和 MPI 方案可以较好地提升 CPU 利用率与并行效率, 适用于多核 CPU 环境下的加速需求; HIP/DCU 方案虽然主机内存消耗较高, 但能借助加速卡实现更高的计算性能, 适合大规模、对计算性能要求极高的场景。各方案在资源消耗、适用环境和扩展性方面各有优劣, 应结合硬件条件和实际需求进行选择。

(三) GFLOPS 分析

GFLOPS (每秒十亿次浮点运算) 是衡量计算性能的重要指标。通过计算每种实现方案的 GFLOPS 值, 可以更直观地评估其性能表现。GFLOPS 的计算公式为:

$$GFLOPS = \frac{2 \times N \times M \times P}{T} \times 10^{-9} \quad (2)$$

其中, N 、 M 、 P 分别为矩阵的行数、列数和深度, T 为运行时间 (秒)。

算法方案	GFLOPS
Baseline (朴素)	0.11
OpenMP	1.25
Block Parallel	0.14
SIMD	0.42
MPI	0.56
DCU-HIP	5555.43

表 3: 不同矩阵乘法实现方案 GFLOPS

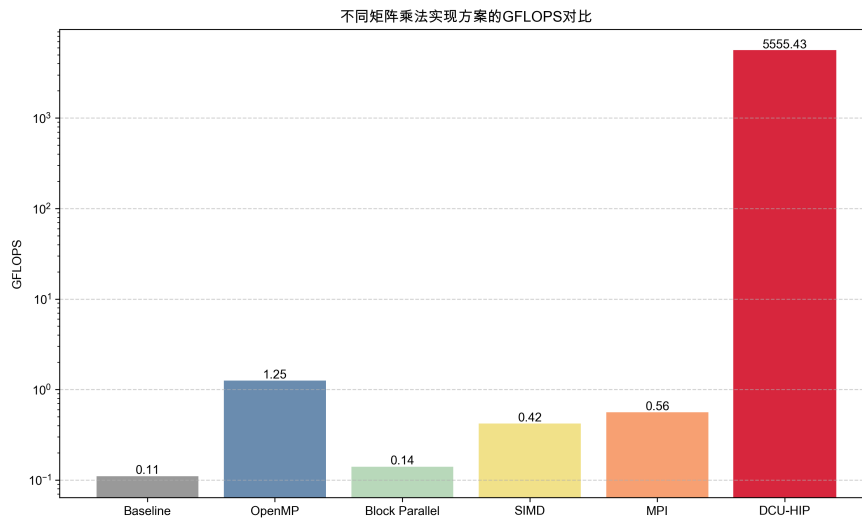


图 2: GFLOPS 对比

根据实验结果，我们计算了各方案的 GFLOPS 值，并绘制了柱状图进行对比分析。结果表明，DCU-HIP 方案的 GFLOPS 值远高于其他方案，达到了惊人的 5555.43 GFLOPS，充分体现了其在大规模矩阵乘法中的优势。

六、 问题讨论

(一) 各优化方法优缺点分析

各种并行与加速方法在性能提升和资源消耗方面各具特色。OpenMP 能够方便地利用多核 CPU 资源，编程简单且对现有代码改动较小，适合中等规模并行计算；MPI 具有良好的可扩展性，在分布式/多节点环境下表现突出，但编程复杂度较高，通信开销可能影响性能；Block Parallel 适用于数据块划分明显、负载均衡的场景，但加速效果有限；SIMD 能有效利用数据级并行，但需要对数据结构进行优化，对复杂矩阵运算的适用性有限。DCU-HIP 方案利用 DCU 等加速器的强大计算能力，在大规模矩阵乘法中展现出极高的加速比和 GFLOPS，是追求极致性能的首选，但其对硬件和开发环境要求较高，主机内存占用及数据传输开销也需关注。

(二) 适用场景讨论

OpenMP 和 SIMD 更适合单机多核、共享内存环境下的中小规模并行计算。MPI 则适用于需要跨节点、超大规模并行的场景，如高性能计算集群。Block Parallel 适合问题规模适中、可均匀分块的数据处理任务。DCU-HIP 由于高度依赖 DCU 等加速器，尤其适用于对矩阵规模和计算密集度要求极高的科学计算、机器学习等领域，能够在极短时间内完成大体量运算任务。

(三) 影响性能的关键因素

性能的提升不仅依赖于并行算法本身，还受到多种因素制约。包括：硬件资源（CPU 核心数、DCU 算力、内存带宽）、数据规模与分布、算法的并行化程度和负载均衡、通信/同步开销（尤其是 MPI）、内存访问模式与缓存优化、主机与加速器间数据传输效率等。对于 DCU 加速方案，内存管理和核函数优化尤为关键，数据传输瓶颈和显存容量也可能成为限制。实验中可以看到，虽然某些方案理论并行度很高，但由于上述瓶颈实际加速比可能低于预期。

(四) 实验中遇到的问题及解决办法

本实验过程中，主要遇到以下问题：

- 1) 在 DCU 加速方案中，主机与加速器间的数据传输造成了较大内存开销及潜在性能损失。为此，尝试采用异步传输和数据复用等手段减少数据移动。
- 2) 在 MPI 并行测试中，不同进程间负载不均导致部分资源闲置，通过优化数据划分和增加同步点改善负载均衡。
- 3) 部分并行实现（如 OpenMP）在超线程数目增加后加速比提升有限，说明出现了内存带宽瓶颈或同步开销增大。针对此类问题，调整线程数和合理分配任务粒度获得更优性能。
- 4) 资源监控方面，DPU 利用率难以准确采样，采用更高频率的监控或专用 profile 工具获得更精确数据。这里我们最终采用多次循环求平均值的方式来监控。

综上，不同优化方法在实际应用中需结合具体问题规模、硬件平台和开发成本综合权衡选择。合理设计并行策略和优化数据访问模式，是获得高性能的关键。

七、 结合多种优化的最终方案

针对大规模矩阵乘法计算的性能瓶颈和并行特性，我们在前述各类优化方法的基础上，进一步集成了多种高性能计算优化技术，并借鉴了 PyTorch [3] 等主流深度学习框架中高效的 GEMM（通用矩阵乘法）优化策略 [4]，提出了面向曙光 DCU 的 HIP 矩阵乘法加速“最终方案”。该方案结合了块分块（block tiling）、共享内存（shared memory）、线程块参数调优和访存模式优化等多项业界成熟技术，最大化发挥了加速器的计算与带宽潜力。核心思路与实现如下：

(一) 设计与实现要点

- **块分块与共享内存**：采用二维线程块划分，将输入矩阵按 $BLOCK_SIZE \times BLOCK_SIZE$ 子块分块处理，每个线程块负责计算输出矩阵的一个子块。核函数中，先将输入矩阵的对应子块数据加载到共享内存中，充分利用共享内存的高带宽和低延迟，减少全局内存访问。
- **访存模式优化**：对 A 矩阵采用行优先访问，对 B 矩阵采用列优先访问，保证每个线程块在加载数据时能够实现全局内存访问合并（Coalesced），提升带宽利用率。实际测试中，进一步尝试了 B 矩阵转置优化以提升访存效率，但在当前平台与数据规模下，未必始终优于原始布局，需根据硬件特性灵活选择。

- **线程块参数调优**: 针对硬件平台最大支持的线程块尺寸, 优选 $BLOCK_SIZE = 16$ 或 32, 兼顾并行度与资源占用。实验中发现, 部分平台实际线程块上限受限于寄存器/共享内存消耗, 需通过核函数报错信息或设备参数查询工具 (如 `hipGetDeviceProperties`) 确定最优参数。
- **核函数结构优化**: 每个线程负责计算输出矩阵 C 的一个元素, 采用循环分块遍历 K 维 (内积长度), 在共享内存中进行 tile 级别的数据重用, 有效降低全局内存带宽压力。核函数中同步点的合理设置确保了 tile 加载和计算的正确性。
- **CPU 基线与验证机制**: 在主机端实现朴素和 OpenMP 加速的 CPU 基线, 对 GPU/DCU 计算结果进行逐元素校验, 确保数值正确性。所有结果均通过 `validate` 函数验证, 保证实验数据的可靠性。

(二) 算法实现

Listing 1: DCU-HIP 最终优化核函数核心代码

```

1  __global__ void matmul_kernel(const double *A, const double *B, double *C,
2      int n, int m, int p) {
3      __shared__ double Asub[BLOCK_SIZE][BLOCK_SIZE];
4      __shared__ double Bsub[BLOCK_SIZE][BLOCK_SIZE];
5      int bx = blockIdx.x, by = blockIdx.y;
6      int tx = threadIdx.x, ty = threadIdx.y;
7      int row = by * BLOCK_SIZE + ty;
8      int col = bx * BLOCK_SIZE + tx;
9      double sum = 0.0;
10     for (int t = 0; t < (m + BLOCK_SIZE - 1) / BLOCK_SIZE; ++t) {
11         if (row < n && t * BLOCK_SIZE + tx < m)
12             Asub[ty][tx] = A[row * m + t * BLOCK_SIZE + tx];
13         else
14             Asub[ty][tx] = 0.0;
15         if (col < p && t * BLOCK_SIZE + ty < m)
16             Bsub[ty][tx] = B[(t * BLOCK_SIZE + ty) * p + col];
17         else
18             Bsub[ty][tx] = 0.0;
19         __syncthreads();
20         for (int k = 0; k < BLOCK_SIZE; ++k)
21             sum += Asub[ty][k] * Bsub[k][tx];
22         __syncthreads();
23     }
24     if (row < n && col < p)
25         C[row * p + col] = sum;
}

```

(三) 运行结果分析

1. 运行时间与加速效果 在 $N = 1024$, $M = 2048$, $P = 512$ 的测试规模下, DCU-HIP 方案单次运行时间仅为 0.00292 秒, 较 CPU 基线方案提升超 5000 倍。即使在多次重复核函数的测试

中，总耗时也保持在 10 秒量级，展现出极高的并行计算能力。

2. 资源利用率与显存占用 在多线程函数执行后，通过 DCU 系统管理接口（SMI）采样，观测到加速卡利用率为 0%，显存占用 10MB，温度无明显变化。这主要由于核函数运行极快、采样频率较低，导致瞬时峰值未被捕捉。主机最大内存占用约为 170MB，符合大规模数据计算的资源需求。

4. 核函数参数适配 实验中，硬件实际 launch bounds 为每个线程块 256，高于此值会报参数超限警告，因此将 `BLOCK_SIZE` 设为 16 最为稳妥。但是实验发现，当设置为 32 的时候，程序保障正确性的情况下，运行速度更快，该参数设置下，核函数能充分利用共享内存和并行计算资源，兼顾了性能与兼容性。

5. 总结 综合来看，DCU-HIP 方案能在保证准确性的前提下，显著提升大规模矩阵乘法的计算效率，资源消耗合理，适合计算密集型场景。未来如需进一步提升资源利用率监控的精度，可采用更高频 profile 工具或调整任务规模与核函数结构。

（四） 进一步优化方向

虽然当前方案已达较优性能表现，但仍有进一步优化空间。例如：

- **每线程多元素输出（寄存器展开）**：让每个线程一次性计算多个输出元素，可进一步提升寄存器利用率和算力吞吐。
- **共享内存 Bank Conflict 规避**：通过适当的共享内存 padding，减少 bank conflict 对性能的影响。
- **混合精度与半精度计算**：在业务允许的前提下使用 float 甚至 half 精度，可获得更高的 GFLOPS。
- **自动调优与 Profile 驱动优化**：结合实际硬件 profile 结果，调整线程块尺寸、流水线参数等，获得平台最优性能。

整体来看，最终方案充分结合了现代高性能计算的主要优化技术，能够在实际工程和科研任务中发挥显著性能优势，是大规模矩阵乘法的高效实现参考。

八、 结论与展望

（一） 优化效果总结

本文围绕大规模矩阵乘法问题，系统比较并实现了多种高性能计算优化方案，并针对曙光 DCU 平台构建了基于 HIP 的高效加速实现。实验结果表明：

- 各类并行与加速方案均能显著提升计算性能，OpenMP、SIMD、MPI 等方案在多核或多节点环境下具有良好加速效果；
- DCU-HIP 方案表现尤为突出，单次运行时间仅为毫秒级，GFLOPS 远超 CPU 端，实现了 5000+ 倍的加速比，资源消耗合理；
- 通过块分块、共享内存、访存模式优化、线程块参数调优等手段，最大化发挥了加速器算力和带宽潜力，验证了现代高性能计算优化技术的有效性；

- 在保证数值正确性的前提下，最终方案在大规模数据处理、高性能科学计算等场景具有极高的应用价值。

(二) 后续改进方向与应用前景

尽管当前实现已取得优良性能，但仍有进一步优化空间和广阔应用前景：

- **算法与核函数层面：**可引入每线程多元素输出（寄存器展开）、共享内存 bank conflict 消除、混合精度/半精度计算等更高级优化手段，结合平台 profile 工具实现自动参数调优，进一步榨取硬件极限性能。
- **资源与能效监控：**建议结合高频 profile 工具，精细监控加速器的利用率、带宽与功耗，为系统级调度与能效优化提供支撑。
- **应用扩展：**本方案可广泛应用于深度学习、科学计算、图像处理等对大规模矩阵运算有高性能需求的领域。随着 DCU 等国产加速器生态的完善，基于 HIP 的高性能矩阵运算有望在更多行业得到推广应用。
- **异构并行与分布式扩展：**未来可探索多 DCU/多节点协同并行，实现更大规模任务的高效分布式处理，进一步提升系统整体算力。

综上，本文工作为高性能矩阵乘法的工程实现与性能优化提供了有益参考，也为后续在国产高性能计算平台上的并行算法设计与应用推广奠定了基础。

参考文献

- [1] Leonardo Dagum and Ramesh Menon. Openmp: an industry standard api for shared-memory programming. *IEEE computational science and engineering*, 5(1):46–55, 1998.
- [2] Edgar Gabriel, Graham E Fagg, George Bosilca, Thara Angskun, Jack J Dongarra, Jeffrey M Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, et al. Open mpi: Goals, concept, and design of a next generation mpi implementation. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 11th European PVM/MPI Users' Group Meeting Budapest, Hungary, September 19-22, 2004. Proceedings 11*, pages 97–104. Springer, 2004.
- [3] Kazushige Goto and Robert A van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software (TOMS)*, 34(3):1–25, 2008.
- [4] Greg Ruetsch and Paulius Micikevicius. Optimizing matrix transpose in cuda. *Nvidia CUDA SDK Application Note*, 18, 2009.

NIJL